

Materiale di studio

Questi documenti spiegano **come funzionano** le cose su cui poggia l'app: le tre tecnologie del browser (01-03), e poi i meccanismi di programmazione che il progetto ha incontrato strada facendo (04 in poi). Non raccontano *cosa fa* PokéDeck (quello lo dicono il codice e i commit): raccontano **il meccanismo** — il ciclo di vita, le API, il modello di esecuzione, l'algoritmo — usando il codice del progetto solo come esempio concreto sotto mano.

#	Documento	La domanda a cui risponde
01	Progressive Web App	Come fa un sito a installarsi e a funzionare senza rete? Cos'è un <i>service worker</i> e come intercetta le richieste?
02	IndexedDB	Com'è fatto un database dentro il browser? Object store, transazioni, versioni dello schema, e perché tutto è asincrono.
03	Web Components	Come si costruisce un componente riutilizzabile senza framework? Custom element, Shadow DOM, ciclo di vita.
04	Immagini che non arrivano	Cosa fa il browser quando un <code></code> fallisce, a cosa serve davvero <code>alt</code> , e come si disegna un segnaposto che eredita il colore dal CSS.
05	Dati che vengono da fuori	Come si aggiunge una chiamata di rete a un'app offline-first senza tradirla: <code>fetch</code> e i suoi errori silenziosi, cache in IndexedDB, migrazioni di schema.
06	Misurare un mazzo	Come si costruisce un punteggio che significhi qualcosa: normalizzare per taglia, calibrare i tetti sui dati, distinguere «zero» da «non lo so», e l'ipergeometrica senza fattoriali.
08	Persistere oggetti	Perché un oggetto salvato non torna indietro uguale? Forma su disco e forma in memoria, <code>null</code> contro assente, e una <code>Promise</code> che aspetta un evento che non arriva.
09	La salita di collina	Come si risolve un problema senza formula? Funzione obiettivo, vicinato, massimi locali — e perché l'algoritmo ottimizza esattamente le stupidaggini che misuri.
10	Riunire due rami	Cosa succede quando due linee di lavoro costruiscono la stessa cosa due volte? Conflitti che Git non segnala, versioni di schema omonime, e perché un test verde può proteggere un difetto.

#	Documento	La domanda a cui risponde
11	L'oggetto incompleto	Se tre punti del codice costruiscono "un mazzo", chi garantisce che sia la stessa cosa? Invarianti senza classi, campi derivabili e non, e perché i test si fabbricano input troppo gentili.
12	Regola o dato?	Della stessa informazione, quale metà si calcola e quale si scarica? Dati che scadono, liste arbitrarie che non si indovinano, e le otto richieste al posto di ventunomila.
13	Stati, rotte e il tasto Indietro	Dove si tiene lo stato di una schermata che mostra tre cose diverse? Routing a frammento con parametri, delega degli eventi per elementi che non esistono ancora, <code><details></code> come fisarmonica nativa — e perché la stampa richiede JavaScript oltre al CSS.
14	Una lingua che manca	La fonte non ha i dati che ti servono, ma li ha in un'altra lingua. Quali campi <i>sembrano</i> testo e in realtà sono chiavi, perché tradurli è obbligatorio e tradurre gli altri sarebbe falso — e una carta che ha fermato ottanta set di download.
15	Un indice per cercare	Cercare fra 21.000 carte senza scaricarle tutte. Quando conviene un indice e quanto può pesare, perché un tetto sui risultati non basta se il costo sta altrove, e un accoppiamento fra due file che nessun compilatore sorveglia.
16	Caricare quando serve	Un interruttore che bloccava la pagina, e i tre costi diversi dietro il blocco. <code>IntersectionObserver</code> invece di <code>scroll</code> , una fila fatta con una promessa, e perché <code>requestAnimationFrame</code> era la scelta sbagliata.
17	Due simboli, un solo angolo	Due funzioni chiedono lo stesso simbolo e lo stesso angolo della card. Quando due stati vogliono un campo solo e quando ne vogliono due, il campo che sparisce a ogni riscrittura del record, un <code><button></code> dentro un <code><button></code> , e una vista che è la stessa griglia con un filtro fisso.

Per chi è scritto

Si dà per scontato che tu conosca **JavaScript e CSS di base**: qui non si spiegano `const`, le classi, il box model. Si dà per scontata anche esperienza di **Java** e **Angular**, e si usa proprio quel bagaglio come pietra di paragone — perché il punto più interessante di queste tecnologie è quasi sempre *in cosa differiscono* da come le stesse cose si fanno lì:

- il service worker è un proxy che gira nel browser — più vicino a un *filtro servlet* che a qualunque cosa di Angular;
- IndexedDB è transazionale come JDBC, ma **asincrono** e **auto-committante**, e questo ribalta abitudini radicate;
- un Web Component ha un ciclo di vita simile a un componente Angular, ma **senza change detection**: se non ti ridisegni a mano, non si ridisegna nessuno.

Come leggerli

Ogni documento è una **sessione di studio**: prima il meccanismo in astratto, poi lo stesso meccanismo nel codice del progetto (`sw.js`, `src/data/deposito.js`, `src/ui/...`), infine qualche domanda di verifica. Si possono leggere in qualunque ordine, ma 01 → 02 → 03 è la progressione pensata.